PRAKTIKUM 1

Sistem Pemantauan dan Kontrol LED

Tujuan :

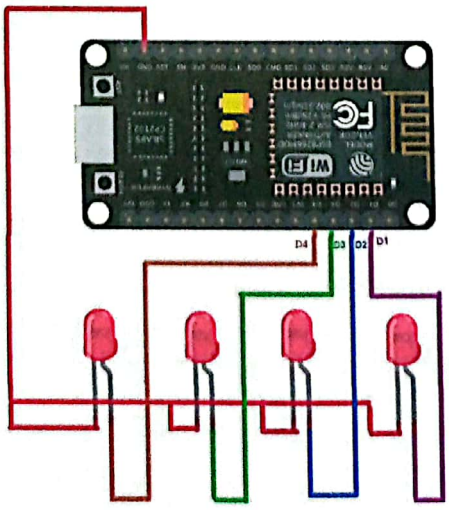
a. Mahasiswa dapat memahami dan menguji konsep Internet of Things (IoT) dengan menggunakan ESP8266.
b. Mahasiswa dapat memahami program kontrol LED berbasis IoT dengan ESP8266.
c. Mahasiswa dapat memahami cara menggunakan Thingspeak untuk memantau kondisi LED.

1.1 Alat Dan Bahan

- *Komputer dan Aplikasi Arduinno IDE*
- *Trainer Kit Teaching AID Mikrokontroler*
- *ESP8266*
- *Kabel USB ESP8266*
- *Kabel Jumper*

1.2 Petunjuk Praktikum

a. Siapkan kebutuhan yang diperlukan praktikum.
b. Hubungkan ESP8266 dengan Laptop menggunakan Kabel USB Arduino.
c. Perhatikan lampu yang ada di Arduino Uno saat percobaan dimulai.
d. Hubungkan kabel Jumper seperti tabel berikut :



Dipindai dengan CamScanner

e. Buat program pada aplikasi Arduino IDE seperti :

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <ThingSpeak.h>
#include <ESP8266HTTPClient.h>
```

Bagian ini adalah impor library yang diperlukan untuk proyek ini. Berikut adalah penjelasan singkat tentang masing-masing library:

- **ESP8266WiFi.h**: Library ini digunakan untuk mengelola koneksi Wi-Fi pada NodeMCU ESP8266.
- **ESP8266WebServer.h**: Ini adalah library yang memungkinkan Anda membuat server web pada NodeMCU untuk mengendalikan dan memonitor perangkat.
- **ESP8266HTTPClient.h**: library ini memungkinkan perangkat ESP8266 untuk berkomunikasi dengan server HTTP, mengirim dan menerima data melalui protokol HTTP, dan melakukan berbagai operasi web seperti mengambil data dari server atau mengirim data ke server.
- **ThingSpeak.h**: Ini adalah library yang memungkinkan Anda mengirim data ke platform IoT ThingSpeak untuk memantau dan menganalisis data Anda.

Selanjutnya, mari kita lihat bagian konfigurasi awal program:

```
const char *ssid = "S23 Ultra";// Ganti dengan nama Wi-Fi Anda
const char *password = "mlrc2551";// Ganti dengan password Wi-Fi
Anda
const char *thingspeakApiKey = "FORQF0VCGTTFH7UO";// Ganti dengan
API Key ThingSpeak Anda
const int thingspeakChannel = 2287952; // Ganti dengan Channel
ThingSpeak Anda

int ledPin1 = D1; // Hubungkan ini ke salah satu terminal motor
int ledPin2 = D2; // Hubungkan ini ke terminal lain motor DC;
int ledPin3 = D3; // Hubungkan ini ke terminal lain motor DC;
int ledPin4 = D4; // Hubungkan ini ke terminal lain motor DC;
WiFiClient client;
ESP8266WebServer server(80);
```

Dipindai dengan CamScanner

Di sini, Anda mengatur beberapa informasi penting:

- **ssid** dan **password**: Ini adalah nama SSID dan kata sandi dari jaringan Wi-Fi yang akan digunakan oleh NodeMCU.
- **writeAPIKey**: Ini adalah kunci API yang digunakan untuk otorisasi pengiriman data ke ThingSpeak.
- **channelID**: Ini adalah ID channel ThingSpeak yang akan digunakan untuk mengirim data.

Selanjutnya, kita akan membuat kode untuk tampilan interface nya.

```
const char *html = R"(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Mengontrol LED</title>
  <style>
    body {
      font-family: 'Arial', sans-serif;
      background-color: #f0f0f0;
      text-align: center;
      padding: 20px;
    }
    h1 {
      color: #333;
    }
    .control-container {
      background-color: #008080;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
      border-radius: 5px;
      display: inline-block;
      padding: 20px;
      max-width: 400px;
      margin: 0 auto;
    }
    .button {
      font-size: 18px;
      padding: 10px 20px;
      background-color: #4CAF50;
      color: #fff;
```

Dipindai dengan CamScanner

```
border: none;
      border-radius: 5px;
      cursor: pointer;
      width: 100%;
      margin: 10px 0;
      text-decoration: none;
    }
    .button:hover {
      background-color: #45a049;
    }
  </style>
</head>
<body>
  <h1>Kontrol LED</h1>
  <div class="control-container">
    <form action='/led' method='POST'>
      <input class="button" type='submit' name='nyala_1'
value='Hidupkan Lampu 1'>
      <input class="button" type='submit' name='mati_1'
value='Matikan Lampu 1' >
      <input class="button" type='submit' name='nyala_2'
value='Hidupkan Lampu 2' >
      <input class="button" type='submit' name='mati_2'
value='Matikan Lampu 2' >
      <input class="button" type='submit' name='nyala_3'
value='Hidupkan Lampu 3'>
      <input class="button" type='submit' name='mati_3'
value='Matikan Lampu 3'>
      <input class="button" type='submit' name='nyala_4'
value='Hidupkan Lampu 4'>
      <input class="button" type='submit' name='mati_4'
value='Matikan Lampu 4'>
    </form>
  </div>
</body>
</html>
)";
```

- `const char *html = R"(` : Kode ini digunakan untuk menyimpan HTML atau teks yang panjang dalam variabel html tanpa harus peduli dengan karakter khusus yang mungkin ada di dalam teks tersebut.

Dipindai dengan CamScanner

Selanjutnya, kita akan melihat fungsi updateThingSpeak

```
void updateThingSpeak(int field, int status) {
  ThingSpeak.begin(client);
  ThingSpeak.setField(field, status);
  ThingSpeak.writeFields(thingspeakChannel, thingspeakApiKey);
}
```

Kode tersebut adalah implementasi dari sebuah fungsi bernama **updateThingSpeak**. Fungsi ini memiliki dua parameter: **field** dan **status**. Fungsi tersebut melakukan hal berikut:

- **ThingSpeak.begin(client)**: Menginisialisasi komunikasi dengan layanan ThingSpeak menggunakan objek **client**.
- **ThingSpeak.setField(field, status)**: Menentukan nilai yang akan dikirim ke bidang (field) yang sesuai dengan nilai **field** yang diberikan.
- **ThingSpeak.writeFields(thingspeakChannel, thingspeakApiKey)**: Mengirim data yang telah disiapkan ke saluran ThingSpeak yang ditentukan dalam **thingspeakChannel** dengan menggunakan kunci API yang disediakan dalam **thingspeakApiKey**.

Selanjutnya, kita akan melihat fungsi **setup()**:

```
void setup() {
  pinMode(ledPin1, OUTPUT);
  pinMode(ledPin2, OUTPUT);
  pinMode(ledPin3, OUTPUT);
  pinMode(ledPin4, OUTPUT);
  digitalWrite(ledPin1, LOW);
  digitalWrite(ledPin2, LOW);
  digitalWrite(ledPin3, LOW);
  digitalWrite(ledPin4, LOW);

  Serial.begin(9600);
  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
```

Dipindai dengan CamScanner

```
}
  Serial.println("Connected to WiFi");
  Serial.println("Alamat IP Lokal: ");
  Serial.println(WiFi.localIP());

  server.on("/", HTTP_GET, []() {
    server.send(200, "text/html", html);
  });

  server.on("/led", HTTP_POST, []() {
      if (server.hasArg("nyala_1")) {
      digitalWrite(ledPin1, HIGH);
      updateThingSpeak(1, 1);
    } else if (server.hasArg("mati_1")) {
      digitalWrite(ledPin1, LOW);
      updateThingSpeak(1, 0);
    } else if (server.hasArg("nyala_2")) {
      digitalWrite(ledPin2, HIGH);
      updateThingSpeak(2, 1);
    } else if (server.hasArg("mati_2")) {
      digitalWrite(ledPin2, LOW);
      updateThingSpeak(2, 0);
    } else if (server.hasArg("nyala_3")) {
      digitalWrite(ledPin3, HIGH);
      updateThingSpeak(3, 1);
    } else if (server.hasArg("mati_3")) {
      digitalWrite(ledPin3, LOW);
      updateThingSpeak(3, 0);
    } else if (server.hasArg("nyala_4")) {
      digitalWrite(ledPin4, HIGH);
      updateThingSpeak(4, 1);
    } else if (server.hasArg("mati_4")) {
      digitalWrite(ledPin4, LOW);
      updateThingSpeak(4, 0);
    }
    server.send(200, "text/html", html);
  });

  server.begin();
}
```

Dipindai dengan CamScanner

setup() ini memiliki beberapa tugas utama:

- **Konfigurasi Pin dan LED**: Menggunakan **pinMode()**, kode mengatur beberapa pin (disimbolkan oleh **ledPin1**, **ledPin2**, **ledPin3**, dan **ledPin4**) sebagai output dan kemudian mengatur inisial statusnya menjadi mati (LOW). Ini berarti LED yang terhubung ke pin tersebut akan dimatikan pada awalnya.
- **Inisialisasi Serial Communication**: Melalui **Serial.begin(9600)**, kode memulai komunikasi serial dengan kecepatan 9600 baud. Ini digunakan untuk debugging dan pencatatan pesan yang akan dikirim ke monitor serial.
- **Konfigurasi dan Koneksi WiFi**: Kode mengatur modus WiFi sebagai mode akses titik (WIFI_AP) dan kemudian membuat jaringan WiFi dengan nama (SSID) dan kata sandi (password) yang telah ditentukan sebelumnya. Selanjutnya, kode menghubungkan perangkat ke jaringan WiFi yang telah ditentukan dalam **while** loop. Selama perangkat belum terhubung (**WiFi.status() != WL_CONNECTED**), maka akan menunggu selama 1 detik (dengan **delay(1000)**) dan mencetak pesan "Connecting to WiFi..." ke monitor serial. Setelah terhubung, kode akan mencetak pesan "Connected to WiFi" dan alamat IP lokal perangkat.
- **Penanganan Permintaan HTTP**: Melalui perintah-perintah di bawah ini, kode menentukan cara menangani permintaan HTTP yang diterima oleh server HTTP yang telah dibuat sebelumnya:
    1. **server.on("/", HTTP_GET, []() { ... })**: Ini menangani permintaan pada akar ("/") dengan metode HTTP GET dan merespons dengan mengirimkan HTML yang ada dalam variabel **html**.
    2. **server.on("/led", HTTP_POST, []() { ... })**: Ini menangani permintaan yang dikirim ke "/led" dengan metode HTTP POST. Ini memeriksa argumen dalam permintaan untuk menentukan apakah harus menyalakan atau mematikan LED (ditunjukkan dengan **nyala_X** dan **mati_X**, dengan X adalah nomor LED). Kemudian, kode akan menyalakan atau mematikan LED sesuai permintaan dan memanggil fungsi **updateThingSpeak** untuk mengirim data ke ThingSpeak. Setelah itu, server akan mengirimkan kembali halaman HTML yang sama ke perangkat klien.
- **Menjalankan Server**: Terakhir, dengan **server.begin()**, kode memulai server HTTP untuk menangani permintaan HTTP dari perangkat lain melalui jaringan WiFi.

Terakhir, kita akan melihat fungsi **loop()**:

```
void loop() {
 server.handleClient();
}
```

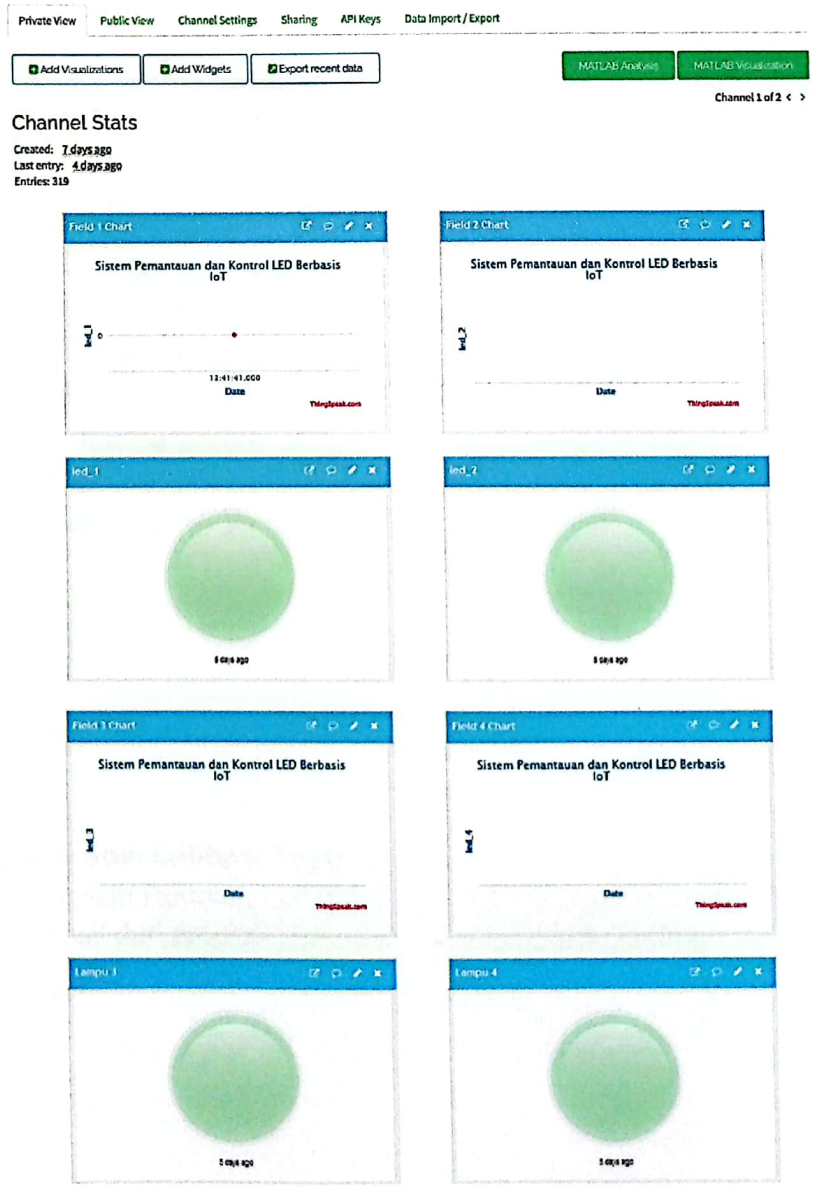
Dipindai dengan CamScanner

Fungsi **loop()** ini hanya melakukan satu hal:

- **server.handleClient()**: Kode ini berfungsi untuk menangani permintaan klien yang datang ke server HTTP yang telah diinisialisasi dalam fungsi **setup()**. Ini memungkinkan server untuk menerima, memproses, dan merespons permintaan HTTP yang dikirim oleh perangkat lain yang terhubung ke perangkat Arduino atau ESP8266 melalui jaringan WiFi.

f. Compile dan Upload program yang telah dibuat.

g. Buatlah Thingspeak yang dibutuhkan praktikum untuk menjadi antarmuka program. Seperti berikut :



Dipindai dengan CamScanner

1.2 Hasil Praktikum

Videokan hasil praktikum pada link dibawah dengan format file "(KELAS)_(TAHUN)" Lalu "(ABSEN_BAWAH/ABSEN_ATAS)" dan hasil video dengan nama "(NAMA/KELOMPOK)_(NIM)_(BAB1)".

1.3 Analisa

- *Dimulai dengan teori yang mendasari praktikum (perhitungan teoritis termasuk disini)*
- *Meresumekan data-data pengukuran*
- *Meresumekan data-data simulasi (pengukuran ideal)*
- *Membandingkan antara teori dengan data yang dihasilkan dengan hasil pengukuran/simulasi*
- *Menerangakan fenomena yang terjadi antara teori dan praktek dan mencari penyebab jika terjadi perbedaan antara teori dan praktek*

1.4 Kesimpulan

Kesimpulan merupakan jawaban atas tujuan

1.5 Studi Kasus/Soal Latihan/Tugas

Buat Program Running Led dengan 4 LED sebagai Output, dua input Start dan Stop. Kemudian hasil dari program tersebut update secara realtime pada ThingSpeak.

Jawaban :

Dipindai dengan CamScanner

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <ThingSpeak.h>
#include <ESP8266HTTPClient.h>

const char *ssid = "S23 Ultra";
const char *password = "mlrc2551";
const char *thingspeakApiKey = "FORQF0VCGTTFH7UO";
const int thingspeakChannel = 2287952;
WiFiClient client;
ESP8266WebServer server(80);
```

Dipindai dengan CamScanner